

Next.js Fundamentals

An interview-focused guide with code examples



App Router vs Pages Router

The architectural shift every interviewer asks about

Next.js 13+ introduced the **App Router** (`app/` directory), which is now the recommended default. The older **Pages Router** (`pages/`) still works and both can coexist in the same project.

FEATURE	PAGES ROUTER	APP ROUTER
Directory	<code>pages/</code>	<code>app/</code>
Default rendering	Client Components	Server Components
Data fetching	<code>getServerSideProps</code> , <code>getStaticProps</code>	async Server Components, <code>fetch</code>
Layouts	Manual <code>_app.tsx</code> wrapper	Nested <code>layout.tsx</code> files
Streaming	Not supported	Built-in with <code>Suspense</code>
Server Actions	Not available	Native support

INTERVIEW TIP

"When would you still choose the Pages Router?" — Legacy migrations, projects already on Pages Router, or when a required library lacks App Router support. Otherwise, always go App Router.

Rendering Strategies

SSR, SSG, ISR, and PPR — know when to use each

Static Site Generation (SSG)

Pages are rendered at build time. Great for blogs, marketing pages, docs.

```
// app/blog/[slug]/page.tsx
export async function generateStaticParams() {
  const posts = await getPosts();
  return posts.map(p => ({ slug: p.slug }));
}

export default async function BlogPost({ params }) {
  const post = await getPost(params.slug);
  return <article>{post.content}</article>;
}
```

Server-Side Rendering (SSR)

Page HTML generated on every request. Use for personalized or real-time content.

```
// Force dynamic rendering
export const dynamic = 'force-dynamic';

export default async function Dashboard() {
  const data = await fetch('https://api.example.com/stats', {
    cache: 'no-store'
  });
  return <StatsView data={data} />;
}
```

Incremental Static Regeneration (ISR)

Static pages that revalidate after a time interval — the best of both SSG and SSR.

```
const data = await fetch('https://api.example.com/products', {
  next: { revalidate: 60 } // Revalidate every 60 seconds
});
```

Partial Prerendering (PPR) — Next.js 14+

Combines static shell with dynamic streamed content in a single route. The static parts are served instantly from CDN, while dynamic parts stream in.

```
export const experimental_ppr = true;

export default function Page() {
  return (
    <main>
      <StaticHeader />           {/* instant */}
      <Suspense fallback=<Skeleton />>
        <DynamicFeed />         {/* streamed */}
      </Suspense>
    </main>
  );
}
```

INTERVIEW TIP

Explain the tradeoffs: SSG = fastest TTFB but stale data; SSR = fresh data but slower TTFB; ISR = middle ground; PPR = best UX by combining both in one response.

File-Based Routing

The filesystem is the API

Every folder in `app/` becomes a URL segment. Special files control behavior:

FILE	PURPOSE
<code>page.tsx</code>	Makes a route publicly accessible
<code>layout.tsx</code>	Shared UI that wraps child routes (persists across navigations)
<code>loading.tsx</code>	Instant loading UI (uses Suspense)
<code>error.tsx</code>	Error boundary for the segment
<code>not-found.tsx</code>	UI for <code>notFound()</code> calls
<code>route.ts</code>	API endpoint (GET, POST, etc.)

Dynamic & Catch-All Routes

```
// Dynamic segment
app/blog/[slug]/page.tsx      → /blog/hello-world

// Catch-all
app/docs/[...slug]/page.tsx   → /docs/a/b/c

// Optional catch-all
app/docs/[...slug]/page.tsx   → /docs OR /docs/a/b
```

Route Groups

Parenthesized folders organize code without affecting the URL:

```
app/  
  (marketing)/  
    about/page.tsx      → /about  
    pricing/page.tsx    → /pricing  
  (dashboard)/  
    layout.tsx          → shared dashboard layout  
    settings/page.tsx   → /settings
```

Parallel Routes & Intercepting Routes

```
// Parallel routes use @named slots  
app/  
  layout.tsx           → receives {children, analytics, team}  
  @analytics/page.tsx  
  @team/page.tsx  
  
// Intercepting routes for modals  
app/  
  feed/page.tsx  
  feed/(.)photo/[id]/page.tsx → intercepts /photo/[id]  
  photo/[id]/page.tsx        → direct visit
```

INTERVIEW TIP

Interviewers love asking about parallel and intercepting routes — they're the mechanism behind Instagram-style photo modals where the URL changes but the feed stays visible behind.

Server & Client Components

The mental model that defines the App Router

In the App Router, all components are **Server Components by default**. They run only on the server, so they can directly access databases, APIs, and the filesystem — and they send zero JavaScript to the client.

```
// Server Component (default – no directive needed)
import { db } from '@/lib/db';

export default async function UserList() {
  const users = await db.user.findMany();
  return (
    <ul>
      {users.map(u => <li key={u.id}>{u.name}</li>)}
    </ul>
  );
}
```

Client Components

Add `'use client'` when you need interactivity, browser APIs, or React hooks.

```
'use client';
import { useState } from 'react';

export default function Counter() {
  const [count, setCount] = useState(0);
  return (
    <button onClick={() => setCount(c => c + 1)}>
      Clicked {count} times
    </button>
  );
}
```

Composition Pattern

Keep the Server Component as the parent, pass data down to Client Components. This minimizes the JS bundle.

```
// app/dashboard/page.tsx (Server)
import { InteractiveChart } from './chart';

export default async function Dashboard() {
  const data = await getAnalytics();
  return <InteractiveChart data={data} />; // data fetched on server
}

// app/dashboard/chart.tsx (Client)
'use client';
export function InteractiveChart({ data }) {
  // useState, onClick, animations – all client-side
}
```

INTERVIEW TIP

"Can a Client Component import a Server Component?" — No. But a Server Component can pass a Server Component as children to a Client Component. This is the "donut pattern."

Data Fetching

No more `getServerSideProps` — just `async/await`

In the App Router, Server Components can be `async`. Use `fetch` directly or call your ORM — Next.js automatically deduplicates identical requests in the same render pass.

```
// Fetching in a Server Component
export default async function ProductPage({ params }) {
  const product = await fetch(
    `https://api.store.com/products/${params.id}`,
    { next: { revalidate: 3600 } } // ISR: cache for 1 hour
  ).then(r => r.json());

  return <h1>{product.name}</h1>;
}
```

Parallel Data Fetching

Avoid waterfalls by fetching in parallel with `Promise.all`:

```
export default async function Page() {
  const [user, posts, notifications] = await Promise.all([
    getUser(),
    getPosts(),
    getNotifications(),
  ]);
  // ...
}
```

Route Handlers (API Routes)

```
// app/api/users/route.ts
import { NextResponse } from 'next/server';

export async function GET(request: Request) {
  const users = await db.user.findMany();
  return NextResponse.json(users);
}

export async function POST(request: Request) {
  const body = await request.json();
  const user = await db.user.create({ data: body });
  return NextResponse.json(user, { status: 201 });
}
```

Key difference: In the Pages Router you'd split fetching between `getServerSideProps` and component rendering. In the App Router, data fetching and rendering are unified in the same async function.

Server Actions

Form mutations without API routes

Server Actions let you define server-side functions and call them directly from Client Components — like RPC. They replace many use cases for API routes.

```
// app/actions.ts
'use server';

import { revalidatePath } from 'next/cache';

export async function addTodo(formData: FormData) {
  const title = formData.get('title') as string;
  await db.todo.create({ data: { title } });
  revalidatePath('/todos');
}
```

```
// app/todos/page.tsx
import { addTodo } from '../actions';

export default function TodoPage() {
  return (
    <form action={addTodo}>
      <input name="title" required />
      <button type="submit">Add</button>
    </form>
  );
}
```

With useActionState (React 19)

```
'use client';
import { useActionState } from 'react';
import { addTodo } from '../actions';

export function TodoForm() {
  const [state, action, isPending] = useActionState(addTodo, null);
  return (
    <form action={action}>
      <input name="title" disabled={isPending} />
      <button>{isPending ? 'Adding...' : 'Add'}</button>
      {state?.error && <p>{state.error}</p>}
    </form>
  );
}
```

INTERVIEW TIP

Server Actions work without JavaScript enabled (progressive enhancement). The form submits as a standard POST, and Next.js handles the rest. This is a strong talking point.

Middleware

Runs before every matched request at the Edge

```
// middleware.ts (project root)
import { NextResponse } from 'next/server';
import type { NextRequest } from 'next/server';

export function middleware(request: NextRequest) {
  const token = request.cookies.get('auth-token');

  if (!token && request.nextUrl.pathname.startsWith('/dashboard')) {
    return NextResponse.redirect(
      new URL('/login', request.url)
    );
  }

  // Add custom headers
  const response = NextResponse.next();
  response.headers.set('x-request-id', crypto.randomUUID());
  return response;
}

export const config = {
  matcher: ['/dashboard/:path*', '/api/:path*'],
};
```

Common middleware uses: authentication guards, redirects, A/B testing, geolocation-based routing, rate limiting, and request logging.

Constraint: Middleware runs on the Edge Runtime — no Node.js APIs like `fs`, no heavy npm packages. Keep it lightweight.

Caching

Next.js has four caching layers — know them all

CACHE LAYER	WHAT IT CACHES	WHERE	OPT OUT
Request Memoization	Duplicate fetch calls in same render	Server, per-request	Use AbortController
Data Cache	fetch responses across requests	Server, persistent	cache: 'no-store'
Full Route Cache	Rendered HTML + RSC payload	Server, persistent	export const dynamic = 'force-dynamic'
Router Cache	RSC payload in browser	Client, session	router.refresh()

Revalidation

```
// Time-based
fetch(url, { next: { revalidate: 60 } });

// On-demand (inside a Server Action or Route Handler)
import { revalidatePath, revalidateTag } from 'next/cache';
revalidatePath('/blog');
revalidateTag('posts');

// Tag a fetch for on-demand revalidation
fetch(url, { next: { tags: ['posts'] } });
```

INTERVIEW TIP

Caching is the most nuanced topic in Next.js. Be ready to explain each layer, when it applies, and how to opt out. Mention that `fetch` in the App Router is extended by Next.js with caching options — it's not the raw browser `fetch`.



Image & Font Optimization

Performance features interviewers love to hear about

next/image

```
import Image from 'next/image';

export default function Hero() {
  return (
    <Image
      src="/hero.jpg"
      alt="Hero image"
      width={1200}
      height={630}
      priority          {/* skip lazy loading for LCP */}
      placeholder="blur" {/* show blurred version while loading */}
    />
  );
}
```

Benefits: automatic WebP/AVIF conversion, responsive `srcSet`, lazy loading by default, layout shift prevention via width/height.

next/font

```
import { Inter, Playfair_Display } from 'next/font/google';

const inter = Inter({ subsets: ['latin'] });
const playfair = Playfair_Display({
  subsets: ['latin'],
  weight: ['400', '700'],
  variable: '--font-playfair',
});

export default function RootLayout({ children }) {
  return (
    <html className={` ${inter.className} ${playfair.variable}`}>
      <body>{children}</body>
    </html>
  );
}
```

Fonts are self-hosted at build time — zero external requests, no layout shift, no FOUT.



Metadata & SEO

Static and dynamic metadata APIs

```
// Static metadata
export const metadata = {
  title: 'My Store',
  description: 'Best products online',
  openGraph: {
    title: 'My Store',
    images: ['/og-image.png'],
  },
};

// Dynamic metadata
export async function generateMetadata({ params }) {
  const product = await getProduct(params.id);
  return {
    title: product.name,
    description: product.description,
  };
}
```

Other SEO features

- `sitemap.ts` — dynamically generate XML sitemaps
- `robots.ts` — generate robots.txt programmatically
- `opengraph-image.tsx` — generate OG images with JSX using `ImageResponse`

Error Handling & Loading UI

Declarative error boundaries and streaming

```
// app/dashboard/loading.tsx
export default function Loading() {
  return <div className="skeleton">Loading dashboard...</div>;
}

// app/dashboard/error.tsx
'use client'; // Error components must be Client Components

export default function Error({
  error,
  reset,
}: {
  error: Error;
  reset: () => void;
}) {
  return (
    <div>
      <h2>Something went wrong</h2>
      <button onClick={() => reset()}>Try again</button>
    </div>
  );
}
```

Streaming with Suspense

```
import { Suspense } from 'react';

export default function Page() {
  return (
    <main>
      <h1>Dashboard</h1>
      <Suspense fallback={<ChartSkeleton />}>
        <SlowChart />      {/* streams in when ready */}
      </Suspense>
      <Suspense fallback={<TableSkeleton />}>
        <SlowTable />      {/* independent stream */}
      </Suspense>
    </main>
  );
}
```

Key insight: Each `<Suspense>` boundary creates an independent streaming chunk. The page shell arrives instantly, and each section fills in as its data resolves — no full-page spinner.

Common Interview Questions

Quick-fire answers to the most frequent questions

Q: HOW DOES NEXT.JS DIFFER FROM REACT?

React is a UI library. Next.js is a full-stack framework built on React that adds routing, server rendering, data fetching, bundling, and deployment optimization out of the box.

Q: WHAT IS THE DIFFERENCE BETWEEN `REDIRECT()` AND `USERROUTER().PUSH()`?

`redirect()` works in Server Components and Server Actions — it throws a special error that Next.js catches to send a 307/303 redirect. `useRouter().push()` is client-side navigation within a Client Component, no server round-trip.

Q: WHEN SHOULD YOU USE ROUTE HANDLERS VS SERVER ACTIONS?

Server Actions are for mutations triggered by forms or UI events. Route Handlers (`route.ts`) are for building REST-style APIs consumed by external clients, webhooks, or when you need fine-grained control over the response (headers, status codes, streaming).

Q: WHAT IS REACT SERVER COMPONENTS (RSC) PAYLOAD?

A compact binary format that describes the Server Component tree. It's sent instead of HTML during client-side navigations, allowing React to reconcile the new UI without losing client state. It's cached in the Router Cache on the client.

Q: HOW DO YOU HANDLE AUTH IN NEXT.JS?

Middleware for route protection (redirect unauthenticated users), Server Components to check session and conditionally render, and Server Actions to handle login/logout mutations. Libraries like NextAuth.js (Auth.js) integrate with all three layers.

Q: WHAT IS USE CACHE (NEXT.JS 15)?

A directive that marks an async function's return value as cacheable. Unlike fetch-level caching, it works with any data source (ORM queries, computations). Pair with `cacheLife()` and `cacheTag()` for fine-grained control.

Q: EXPLAIN THE RENDERING ORDER OF A NEXT.JS REQUEST.

1. Middleware runs first (Edge). 2. Route is matched to a segment. 3. Layouts render top-down (cached if unchanged). 4. The page Server Component renders, fetching data. 5. HTML streams to the client with RSC payload. 6. Client Components hydrate. 7. Suspense boundaries resolve as async data arrives.

Q: WHAT IS UNSTABLE_CACHE VS FETCH CACHE?

`fetch cache` is automatic for HTTP requests. `unstable_cache` (now being replaced by `use cache`) wraps any async function — database calls, computations — in the Data Cache with tags and revalidation support.

